BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no:43 PRN No:12410889 Name of Student: Akshada Ganesh Sadaphal

Subject Name: Deep Learning

Problem Statement: Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

- Start.
- Import required libraries.
- Load and preprocess the dataset.
- Split the dataset into training and testing sets.
- Build the MLP model.
- Compile the model with Adam optimizer.
- Train the model using model.fit().
- Evaluate the model on test data.
- Plot accuracy and loss graphs.
- Repeat for different learning rates and epochs.
- Compare the results.
- Stop.

[1] ✓ 6s

```
!pip install tensorflow
```

Show hidden output

[2] ✓ 4s

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

[3] ✓ 0s

```
iris = load_iris()

X = iris.data
y = iris.target
```

[4] ✓ 0s

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

[5] ✓ 0s

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[6] ✓ 0s

```
model = Sequential()

model.add(Dense(16,
                activation='relu',
                input_shape=(4,)))

model.add(Dense(8,
                activation='relu'))

model.add(Dense(3,
                activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[14] ✓ 4s

```
model.fit(
    X_train,
    y_train,
    epochs=50
)
```

Show hidden output

[8] ✓ 0s

```
optimizer = Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

[9] ✓ 4s

```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    validation_split=0.2,
    verbose=1
)
```

Show hidden output

[10] ✓ 0s

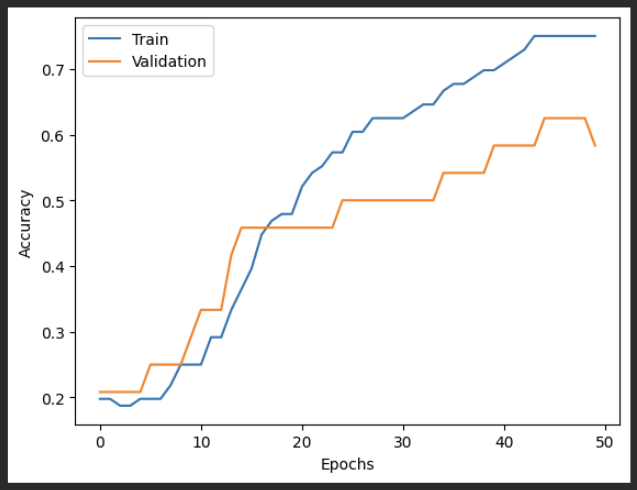
```
loss, accuracy = model.evaluate(X_test, y_test)

print("Accuracy:", accuracy)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - accuracy: 0.7667 - loss: 0.6743
Accuracy: 0.7666666507720947
```

[16]
✓ 0s

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train","Validation"])
plt.show()
```



[15]
✓ 0s

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train","Validation"])
plt.show()
```

